

Trilha de Node.js

Módulo 08: Apis Restful e MySQL

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.
2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.
3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.
4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.
5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.
6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.
7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.
8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 8: APIs RESTful com Node.js e MySQL

Objetivo:

Capacitar o aluno a projetar, estruturar e implementar APIs RESTful utilizando Node.js, Express e MySQL, aplicando boas práticas de organização de código, validação de dados e comunicação com banco de dados.

Ao final do módulo, o aluno será capaz de:

- Criar APIs seguindo padrões REST
 - Integrar aplicações Express com MySQL
 - Implementar operações CRUD completas
 - Aplicar validação de dados no backend
 - Organizar projetos em camadas (routes, controllers e models)
 - Documentar APIs de forma profissional (ex.: Swagger)
-

8.1: Estruturando uma API RESTful com Express

O que é uma API RESTful?

API (Application Programming Interface)

É uma interface que permite a comunicação entre diferentes sistemas e aplicações, possibilitando o envio e recebimento de dados de forma padronizada.

RESTful

Refere-se a APIs que seguem o padrão arquitetural REST (Representational State Transfer), utilizando:

- Verbos HTTP (GET, POST, PUT, DELETE)
- URLs bem definidas e organizadas
- Respostas padronizadas, geralmente em JSON

Por que usar APIs RESTful?

APIs RESTful são amplamente utilizadas porque:

- Facilitam a integração entre sistemas
- São simples e escaláveis
- Utilizam padrões da web já consolidados
- São ideais para aplicações web e mobile

Estrutura do Projeto

Para organizar melhor a aplicação e facilitar a manutenção do código, é recomendado separar o projeto em camadas.

Crie as seguintes pastas e arquivos:

└─ config/	# Configurações (ex.: conexão com banco)
└─ controllers/	# Lógica das requisições e regras de negócio
└─ middlewares/	# Middlewares globais (auth, logger, erros)
└─ models/	# Acesso ao banco e queries SQL
└─ routes/	# Definição dos endpoints da API
└─ validators/	# Validação de dados (Joi etc.)
└─ node_modules/	# Dependências instaladas pelo npm
└─ .env	# Variáveis de ambiente e segredos
└─ .env.example	# Exemplo de configuração do .env
└─ app.js	# Arquivo principal que inicia o Express
└─ schema.sql	# Script de criação do banco/tabelas
└─ swagger.json	# Configuração da documentação Swagger
└─ package.json	# Dependências e scripts do projeto
└─ package-lock.json	# Versões exatas das dependências
└─ README.md	# Documentação do projeto

Essa organização ajuda a:

- Separar responsabilidades
- Manter o código mais limpo
- Facilitar manutenção e escalabilidade
- Seguir boas práticas de mercado

Exemplo de Configuração do Servidor

```
require('dotenv').config();
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  port: Number(process.env.DB_PORT || 3306),
  waitForConnections: true,
  connectionLimit: 5
});

module.exports = pool;
```

8.2: Criando endpoints para CRUD no MySQL

Configuração de Rotas

Antes de prosseguir, é recomendável organizar os endpoints da aplicação em um arquivo específico de rotas. Isso melhora a organização do projeto e facilita a manutenção do código.

Abaixo temos um exemplo de arquivo de rotas centralizando os endpoints de usuários:

```
const express = require('express');
const controller = require('../controllers/usuariosController');

const router = express.Router();

// CRUD
router.post('/usuarios', controller.create);
router.get('/usuarios', controller.read); // paginação + filtro ?page=&limit=&nome=
router.get('/usuarios/:id', controller.readById);
router.put('/usuarios/:id', controller.update);
router.delete('/usuarios/:id', controller.delete);

module.exports = router;
```

Modelo de Dados

- Exemplo de estrutura de tabela no MySQL:

```
CREATE DATABASE IF NOT EXISTS seguranca_demo;
USE seguranca_demo;

DROP TABLE IF EXISTS usuarios;

CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  senha VARCHAR(255) NOT NULL
);

INSERT INTO usuarios (nome, email, senha) VALUES
('Admin', 'admin@demo.com', '123456'),
('User', 'user@demo.com', '123456');
```

Endpoints para CRUD

Agora, na pasta **controllers**, implementamos os métodos responsáveis por tratar as requisições.

Os controllers:

- Recebem os dados da requisição
- Validam as informações

- Chamam as funções do model
- Retornam a resposta ao cliente

Essa etapa é responsável por conectar as rotas às regras de negócio da aplicação.

```
const Usuarios = require('../models/usuariosModel');
const { criarUsuarioSchema, atualizarUsuarioSchema } = require('../validators/usuarioSchema');

function validar(schema, body) {
  const { error, value } = schema.validate(body, { abortEarly: true, stripUnknown: true });

  if(error) {
    const msg = error.details?.[0]?.message || 'Dados inválidos';
    const err = new Error(msg);
    err.status = 400;
    throw err;
  }

  return value;
}
```

- Create (POST)

```
exports.create = async (req, res, next) => {
  try {
    const dados = validar(criarUsuarioSchema, req.body);
    const user = await Usuarios.criar(dados);
    return res.status(201).json({ ...user, message: 'Usuário criado com sucesso!' });
  } catch (err) {
    return next(err);
  }
};
```

- Read (GET)

```
exports.read = async (req, res, next) => {
  try {
    const { page, limit, nome } = req.query;
    const data = await Usuarios.listar({ page, limit, nome });
    return res.json(data);
  } catch (err) {
    return next(err);
  }
};
```

- Update (PUT)

```
exports.update = async (req, res, next) => {
  try {
    const id = Number(req.params.id);
    const dados = validar(atualizarUsuarioSchema, req.body);

    const affected = await Usuarios.atualizar(id, dados);
    if(!affected) return res.status(404).json({ error: 'Usuário não encontrado' });

    return res.json({ message: 'Usuário atualizado com sucesso' });
  } catch (err) {
    return next(err);
  }
};
```

1. Delete (DELETE)

```
exports.delete = async (req, res, next) => {
  try {
    const id = Number(req.params.id);
    const affected = await Usuarios.remover(id);

    if(!affected) return res.status(404).json({ error: 'Usuário não encontrado' });

    return res.json({ message: 'Usuário excluído com sucesso' });
  } catch (err) {
    return next(err);
  }
};
```

8.3: Validação de dados com bibliotecas como Joi

O que é Joi?

- Uma biblioteca para validação de esquemas e dados recebidos via requisições.

Instalação

```
npm install joi
```

Exemplo de Uso

```
const Joi = require('joi');

const criarUsuarioSchema = Joi.object({
  nome: Joi.string().min(3).required(),
  email: Joi.string().email().required(),
  senha: Joi.string().min(6).required()
});

const atualizarUsuarioSchema = Joi.object({
  nome: Joi.string().min(3),
  email: Joi.string().email(),
  senha: Joi.string().min(6)
}).min(1); // exige pelo menos 1 campo no body

module.exports = {
  criarUsuarioSchema,
  atualizarUsuarioSchema
};
```

Agora nas chamadas de criação ou atualização de usuários realizamos a utilização desta função que demos como **validar**, segue o exemplo abaixo:

```
const dados = validar(criarUsuarioSchema, req.body);
```

8.4: Implementando paginação e filtros nas consultas

```
async function listar({ page = 1, limit = 10, nome }) {
  const p = Math.max(parseInt(page, 10) || 1, 1);
  const l = Math.min(Math.max(parseInt(limit, 10) || 10, 1), 100);
  const offset = (p - 1) * l;

  const where = nome ? 'WHERE nome LIKE ?' : '';
  const params = [];

  if(nome) params.push(`%${nome}%`);
  params.push(l, offset);

  const [rows] = await pool.execute(
    `SELECT id, nome, email FROM usuarios ${where} ORDER BY id DESC LIMIT ? OFFSET ?`,
    params
  );

  return { page: p, limit: l, results: rows };
}
```

O que é Paginação?

Paginação é a técnica de dividir os resultados de uma consulta em **partes menores (páginas)**.

Em vez de retornar todos os registros do banco de uma vez, a API retorna apenas uma quantidade limitada por vez.

Por que usar paginação?

- Evita sobrecarregar o servidor
- Reduz tráfego de dados
- Melhora performance
- Facilita navegação de grandes volumes de dados

Como a paginação funciona no código?

1 Definição de página e limite

```
async function listar({ page = 1, limit = 10, nome }) {
```

Exemplo:

```
//?page=2&limit=10, Retorna do registro 11 ao 20.
```

2 Cálculo do OFFSET

```
const offset = (p - 1) * l;
```

OFFSET indica ao banco de onde começar a leitura.

O que é Filtro?

Filtro permite buscar dados específicos com base em um critério.

No exemplo:

```
const where = nome ? 'WHERE nome LIKE ?' : '';
const params = [];

if(nome) params.push(`%${nome}%`);
params.push(1, offset);
```

Se o usuário informar um nome, a consulta será filtrada.

Exemplo de filtro por nome

```
//?nome=jo  
//Consulta executada: WHERE nome LIKE '%jo%'  
//Retorna apenas usuários que possuem "jo" no nome.
```

8.5: Documentação da API com Swagger

O que é Swagger?

Swagger é um conjunto de ferramentas utilizadas para **documentar, testar e visualizar APIs RESTful** de forma interativa.

Ele permite que desenvolvedores entendam e utilizem a API sem precisar ler o código-fonte.

Swagger é baseado no padrão **OpenAPI Specification (OAS)**, que define como descrever endpoints, parâmetros e respostas de uma API.

Por que documentar APIs?

Documentar APIs é importante porque:

- Facilita o uso da API por outros desenvolvedores
- Reduz dúvidas sobre endpoints e parâmetros
- Permite testes diretos no navegador
- Padroniza a comunicação entre equipes
- Torna o projeto mais profissional

👉 Em empresas, APIs sem documentação são consideradas más práticas.

O que o Swagger oferece?

Com Swagger é possível:

- ✓ Visualizar todos os endpoints da API
- ✓ Ver parâmetros aceitos
- ✓ Ver exemplos de respostas
- ✓ Testar requisições diretamente na interface
- ✓ Simular chamadas sem precisar de Postman

Como funciona?

A API é descrita em um arquivo **swagger.json** (ou YAML) contendo:

- rotas (endpoints)
- métodos HTTP (GET, POST, etc.)

- parâmetros
- respostas esperadas
- exemplos de payload

O Swagger-UI lê esse arquivo e gera uma interface web automática.

Instalação do Swagger-UI

```
npm install swagger-ui-express
```

Exemplo de Configuração

```
const swaggerUi = require('swagger-ui-express');
const swaggerDocument = require('./swagger.json');

// Swagger
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument));
```

 **Observação:** Consulte aqui o material de exemplo com práticas para treinar suas habilidades: [Módulo 8](#)

Lista de Exercícios

Questões Teóricas

1. O que caracteriza uma API RESTful?
2. Quais são os verbos HTTP mais usados em APIs e suas funções?
3. Explique a vantagem de estruturar projetos em pastas como **controllers**, **models** e **routes**.
4. O que é Joi e para que serve?
5. Como a paginação melhora a performance de uma API?
6. Qual é a finalidade do Swagger em projetos de API?
7. O que é o conceito de filtros em consultas?
8. Explique como implementar um endpoint seguro para criar registros no MySQL.
9. Quais são os principais benefícios de validar dados na camada da API?
10. Como o Express simplifica o desenvolvimento de APIs RESTful?

Questões Práticas

1. Estruture um projeto para uma API RESTful com as pastas adequadas.
2. Implemente um endpoint para listar todos os registros de uma tabela.
3. Crie um endpoint para inserir um novo registro no banco de dados, validando os dados com Joi.

4. Adicione paginação a um endpoint que lista registros.
5. Crie um endpoint que permita atualizar registros pelo ID.
6. Implemente um endpoint para excluir registros com base no ID.
7. Configure um filtro para buscar registros pelo nome em uma tabela.
8. Documente sua API com Swagger, incluindo os endpoints criados.
9. Teste a validação de dados no backend enviando requisições com dados inválidos.
10. Configure um middleware global que registre os logs de cada requisição recebida.